

ExitAI -- Code Line by Line

A complete walkthrough of every code file in the repository. Each meaningful line or block is shown followed by a plain-English explanation. Generated 2026-09-21.

1 . Folder Structure

Exit Ai/
agents/ Python multi-agent backend (LangGraph hub-and-spoke)
hub/ Orchestrator -- supervisor.py drives the full pipeline
spokes/ Individual agents (HR, IT, Finance, Risk, Docs, ...)
core/ Shared utilities: LLM calls, tracing, email, config
analytics/ Batch analytics agents (run manually, no scheduler)
service.py Local HTTP bridge (port 8787) -- frontend -> pipeline
run_case.py CLI entry point: run the full pipeline for one case
e2e_test.py Smoke test -- direct pipeline calls, no test framework
oauth_setup.py One-time Google OAuth flow for Calendar booking
src/ React + Vite frontend
App.jsx Root: auth check, role detection, renders dashboard
main.jsx React entry point -- mounts <App> in <StrictMode>
routes/ Role-based page components
employee/ Employee dashboard, tasks, documents, exit interview
hr/ HR dashboard: all exits, risk, compliance, reports
manager/ Manager dashboard: KT approvals, clearances
it/ IT dashboard: deprovisioning, asset recovery
finance/ Finance dashboard: dues settlement
shared/ Shared pages: Help, Placeholder
components/ Reusable UI components (Login, Sidebar, PageHead...)
lib/ Frontend helpers: supabase client, status formatters
styles/ CSS design tokens and dashboard styles
supabase/
migrations/ 0001-0030: authoritative Postgres schema source
functions/ Supabase Edge Functions (Deno/TypeScript, serverless)
submit-resignation/ Creates an exit_cases row when employee resigns
ask/ RAG assistant: embed -> retrieve -> LLM -> answer
forward-to-hr/ Emails forwarded question to HR inbox
scripts/
ingest_docs.js Chunks exit_policy.md and embeds into exit_docs
seed/seed.js Seeds 103 demo profiles + 15 exit cases
seed/seed_delegates.js Seeds one delegate per role (hr/manager/it)

2 . Code Files -- Line by Line

agents/core/config.py

Reads all secrets from .env and creates the single shared Supabase client used by every Python agent.

```
from __future__ import annotations
```

> Concept: 'from __future__ import annotations' -- makes all type hints strings at runtime so Python 3.9 can understand newer syntax like 'str | None' without crashing.

```
import os from pathlib import Path
```

Imports the standard library modules for reading environment variables (os) and working with file paths in an OS-independent way (pathlib.Path).

```
from dotenv import load_dotenv from supabase import Client, create_client
```

> Concept: import -- brings code from external packages into this file. 'dotenv' reads a .env file into environment variables; 'supabase' is the Supabase Python SDK.

```
load_dotenv(Path(__file__).resolve().parent.parent.parent / ".env", override=True)
```

Loads the root .env file. `__file__` is this file's path; `.parent.parent.parent` walks up three folders to the repo root. `override=True` forces the .env values even if the same variable already exists in the system environment -- needed because a stale Windows system env var was shadowing the correct gateway URL.

```
SUPABASE_URL = os.environ["SUPABASE_URL"]
```

Reads SUPABASE_URL from the environment (already loaded by `load_dotenv` above). Uses `os.environ["KEY"]` -- a `KeyError` if the variable is missing -- rather than `os.environ.get()`, so a missing secret fails loud instead of silently using `None`.

```
SUPABASE_SERVICE_KEY = os.environ["SUPABASE_SERVICE_KEY"]
```

The service-role key bypasses Postgres row-level security (RLS). It is server-side only -- never sent to the browser.

```
ANTHROPIC_BASE_URL = os.environ["ANTHROPIC_BASE_URL"]
```

The Portkey gateway URL. Named `ANTHROPIC_*` because the wire protocol is the Anthropic Messages API, even though the actual model is Azure OpenAI (Portkey translates behind the scenes).

```
ANTHROPIC_API_KEY = os.environ["ANTHROPIC_API_KEY"]
```

The Portkey API key. Same naming convention -- describes the protocol, not the vendor.

```
ANTHROPIC_MODEL = os.environ["ANTHROPIC_MODEL"]
```

The model string sent in every LLM request. Currently `@azure-openai-eus2/gpt-5.6-sol`.

```
GMAIL_ADDRESS = os.environ["GMAIL_ADDRESS"] GMAIL_APP_PASSWORD = os.environ["GMAIL_APP_PASSWORD"]
```

Gmail credentials used by the notification system to send emails via SMTP. The app password is a Google-generated token, not the account password.

```
EMAIL_TEST_RECIPIENT = os.environ.get("EMAIL_TEST_RECIPIENT")
```

Optional. When unset (`None`) all emails are only logged to the terminal; when set, real SMTP sends go out. The `.get()` here is intentional -- this is a dev-safety gate, so absence is the safe default.

```
GOOGLE_CLIENT_ID = os.environ.get("GOOGLE_CLIENT_ID")
```

Optional Google OAuth credentials for the Calendar booking feature. All use `.get()` so they return `None` when not configured -- `calendar_booking.py` treats `None` as 'log only'.

```
KT_CALENDAR_ID = os.environ.get("KT_CALENDAR_ID", "primary")
```

Which Google Calendar to write KT events to. Defaults to "primary" (the account's main calendar) if not set.

```
db: Client = create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)
```

Creates the one shared Supabase client object. Every agent imports this `db` object and calls `db.table("...").select().execute()` etc. The service key means it can read/write any row regardless of RLS policies.

agents/core/trace.py

Live terminal trace: prints every LLM call (model + latency + tokens), every DB write, and agent start/done as they happen. Used by every agent via `@traced_node` and `log_llm/log_db`.

```
TRACE_LEVEL = os.getenv('TRACE_LEVEL', 'full') # full | summary | off
```

Reads the `TRACE_LEVEL` environment variable. Default 'full' prints everything. 'summary' prints only start/done. 'off' skips all trace output. Lets you silence the trace in production without code changes.

```
REDACT = {"password", "ssn", "email", "token", "api_key"}
```

Set of field names that should never be printed in trace output. Any dict key matching one of these gets replaced with `***` to prevent PII from appearing in terminal logs.

```
_C = {"reset": "\033[0m", "dim": "\033[2m", "bold": "\033[1m", ...}
```

ANSI escape codes for terminal colors. `\033[` is the escape sequence; the number picks the color. These only work in a real terminal, not a log file.

```
def _c(text: str, color: str) -> str: if not sys.stdout.isatty(): return text return f"_{C[color]}{text}_{C['reset']}"
```

Wraps text with a color code, but only when output is going to a real interactive terminal (`sys.stdout.isatty()`). If `stdout` is a file or pipe, returns the plain text unchanged -- so saved logs are clean.

```

_depth: ContextVar[int] = ContextVar("trace_depth", default=0)
> Concept: ContextVar -- a Python variable that has a different value for each concurrent thread/async task. Tracks how deeply nested the current call is (supervisor calls hr_agent calls llm -> depth 2). Used to indent trace output.

print(f"{'| ' * _depth.get()}{line}", flush=True)
Prints the trace line prefixed by | repeated once per nesting level. flush=True makes the output appear immediately instead of buffering -- essential for live trace during long LLM calls.

def traced_node(name: str):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(state, *args, **kwargs):
            > Concept: decorator factory -- a function that returns a decorator. @traced_node("HR agent") wraps a function so that every time it is called, the wrapper runs first (printing start, timing), then calls the original function, then prints done/failure.

            start = time.perf_counter()
            Records the wall-clock time before the function runs. perf_counter() is a high-resolution timer, more accurate than time.time() for measuring short durations.

            token = _depth.set(_depth.get() + 1)
            Increments the indent depth for this call and saves a "reset token" to undo it afterward. Using ContextVar.set() returns a token so you can restore the previous value even if an exception occurs.

            result = fn(state, *args, **kwargs)
            Calls the original (wrapped) function with all its arguments. If this raises an exception, the except block below catches it and prints the failure before re-raising.

            _depth.reset(token)
            Restores the indent depth to what it was before this call -- even after exceptions, because it appears in both the try and except paths.

            elapsed = time.perf_counter() - start
            Calculates how long the function took in seconds, as a float.

            def log_llm(model: str, call):
                start = time.perf_counter()
                resp = call()
                elapsed = time.perf_counter() - start
                Times an LLM call. The call argument is a zero-argument lambda (e.g. lambda: _post(system, user, tokens)). Calling call() here triggers the actual HTTP request, and the elapsed time measures exactly that network round-trip.

            _emit(_c(f"-> llm {model}", "magenta") + _c(f" {elapsed:.2f}s {_tokens(resp)}", "dim"))
            Prints the LLM trace line: model name in magenta, then latency and token counts dimmed. -> is a branch arrow showing it is a sub-step inside the current traced_node.

            def log_db(action: str, table: str, rows=None, detail="") -> None:
                Prints a database write trace line. Called after every db.table(...).insert/update/upsert. Never makes the DB call itself -- purely a logging function.

            def _tokens(resp) -> str:
                usage = getattr(resp, "usage", None)
                Extracts token counts from the LLM response. Uses getattr() because the response object's shape differs between the Anthropic SDK (resp.usage.input_tokens) and LangChain (resp.usage_metadata).

```

agents/core/llm.py

Single point for all LLM calls: plain HTTP POST to the Portkey gateway using the Anthropic Messages wire format. Returns the text response or a parsed JSON dict.

```

import requests
> Concept: the 'requests' library -- makes HTTP calls from Python. Used instead of the Anthropic SDK because the gateway (Portkey) needs plain HTTP with specific headers, not the SDK's own base_url plumbing.

def _post(system: str, user: str, max_tokens: int) -> SimpleNamespace:
    Internal function. Takes a system prompt (instructions for the model) and a user message, sends them to the gateway, returns the response wrapped in a SimpleNamespace.

    resp = requests.post(f"{ANTHROPIC_BASE_URL.rstrip('/')}/v1/messages",
        Makes an HTTP POST to the /v1/messages endpoint. .rstrip("/") removes any trailing slash from the base URL so it doesn't create double slashes.

    headers={ "Authorization": f"Bearer {ANTHROPIC_API_KEY}", "anthropic-version": "2023-06-01",

```

The three required headers for the Anthropic Messages API wire format. Portkey reads these and forwards them to the actual Azure OpenAI backend while translating the request body.

```
json={ "model": ANTHROPIC_MODEL, "max_tokens": max_tokens, "system": system, "messages": [{"role": "user", "content": user}], },
```

The request body. "system" is the instructions; "messages" is the conversation with one user turn. max_tokens limits how many tokens the model can output -- important for cost and for preventing the reasoning model from running out of budget.

```
resp.raise_for_status()
```

If the HTTP status code is 4xx or 5xx, raise_for_status() throws a requests.HTTPError exception immediately. This prevents the code below from trying to parse an error response as a valid model reply.

```
text_block = next((b for b in body["content"] if b.get("type") == "text"), None)
```

> Concept: generator expression -- (expression for item in iterable if condition) creates a lazy iterator. next(..., None) gets the first matching item or None if there are none. This searches for the text block because reasoning models (like GPT-5.6) emit a "thinking" block first, so body["content"][0] is not always the text.

```
if text_block is None: raise RuntimeError(...)
```

If no text block was found (can happen when the model uses all its tokens on reasoning before producing output), raise a clear error message explaining what happened and what to do (raise max_tokens).

```
return SimpleNamespace( content=[SimpleNamespace(text=text_block["text"])],  
usage=SimpleNamespace(**body.get("usage", {})), )
```

> Concept: SimpleNamespace -- a lightweight object that lets you access dict keys as attributes (resp.usage.input_tokens instead of resp["usage"]["input_tokens"]). Wraps the response so trace.log_llm can use attribute access on it.

```
def ask_claude(system: str, user: str, max_tokens: int = 500) -> str:
```

Public function used by every agent that needs a text response. Calls _post through log_llm (which times it and prints the trace), then returns just the text string.

```
def ask_claude_json(system: str, user: str, max_tokens: int = 500) -> dict:
```

Same as ask_claude but parses the response as JSON. Used when the system prompt tells the model to return a JSON object.

```
match = re.search(r"\{.*\}", text, re.DOTALL)
```

> Concept: re.search -- finds the first occurrence of a pattern in a string. r"\{.\}" matches anything between { and }. re.DOTALL makes . also match newlines (so multi-line JSON objects are captured). This tolerates the model wrapping its JSON in ```json ... ``` markdown fences.*

```
return json.loads(match.group(0) if match else text)
```

json.loads() parses the JSON string into a Python dict. Uses match.group(0) (the matched substring) if a JSON block was found, otherwise tries to parse the whole text directly.

agents/core/prompts.py

Single function: reads a prompt file from the agents/core/prompts/ directory by relative path.

```
_ROOT = Path(__file__).resolve().parent.parent.parent / "prompts"
```

Computes the absolute path to the prompts/ directory at the repo root. __file__ is this file (prompts.py); .parent three times reaches the repo root; / "prompts" appends the folder name.

```
def load_prompt(rel_path: str) -> str: return (_ROOT /  
rel_path).read_text(encoding="utf-8").rstrip("\n")
```

Reads a .md prompt file and returns its text. .rstrip("\n") removes any trailing newline so the prompt doesn't have extra whitespace when concatenated with user content. Called at module load time (as a constant), so the file is read once, not on every LLM call.

agents/core/notifications.py

All email sending for the platform: SMTP via Gmail. Provides one function per email type. Dev-safe: no real sends unless EMAIL_TEST_RECIPIENT is set.

```
import smtplib from email.mime.text import MIMEText
```

Standard library email modules. smtplib handles the SMTP connection and sending; MIMEText wraps the email body in the MIME format email servers expect.

```
SENDER_DISPLAY = 'ExitAI (Perficient)' SIGNOFF = ('Regards,', 'The ExitAI Team', 'Perficient')
```

Constants used in every outgoing email so all messages share the same sender name and closing lines.

```
def _compose(to_name, intro, lines=(), closing="") -> str:
```

Pure function (no I/O): builds the plain-text email body. Takes a greeting name, intro sentence, optional bullet lines, and a closing sentence. Returns a formatted string. Every email template calls this instead of building its own body.

```
greeting = f"Hi {to_name}," if to_name else "Hi there,"
```

Uses a conditional expression (value_if_true if condition else value_if_false) to greet by name when known, or generically when not.

```
parts += [""] + [f" - {l}" for l in lines]
```

> Concept: list comprehension -- [expression for item in iterable] builds a new list. This turns each task title into a "- title" bullet point line.

```
@traced_node("Notification agent -- send email") def _send(to: str, subject: str, body: str) -> dict:
```

Internal send function, decorated with @traced_node so it appears in the live terminal trace.

```
if not EMAIL_TEST_RECIPIENT: print(f"[email:dev-log] to={to}\nsubject={subject}\n{body}\n") return {"sent": False, "logged": True, "to": to}
```

Dev-safety gate. When EMAIL_TEST_RECIPIENT is not set (the default), prints the email to the terminal and returns immediately -- no SMTP connection is made. This means running locally never accidentally emails real people.

```
with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp: smtp.login(GMAIL_ADDRESS, GMAIL_APP_PASSWORD) smtp.sendmail(GMAIL_ADDRESS, [to], msg.as_string())
```

> Concept: context manager (with statement) -- automatically closes the SMTP connection when the block exits, even on error.

SMTP_SSL on port 465 opens an encrypted connection immediately (unlike STARTTLS). smtp.login authenticates; smtp.sendmail sends the message.

```
def send_kt_reminder(case: dict, tasks: list[dict]) -> dict:
```

Sends a knowledge-transfer reminder to both the employee and their manager. Takes the full case dict and a list of KT task dicts. Calls _send for each recipient.

```
def check_overdue_and_notify() -> dict:
```

Scans the database for pending tasks past their due_date and sends one warning email per task. Run manually or on a cron -- no automatic scheduler exists in this repo.

```
overdue = db.table("exit_tasks").select(...).eq("status", "pending").lt("due_date", today).execute().data or []
```

> Concept: Supabase query chain -- each .method() call adds a filter or selector. .lt("due_date", today) means "due_date less than today". .execute() sends the query; .data is the list of matching rows (or [] if None).

agents/core/calendar_booking.py

Books Google Calendar events for KT sessions. Dev-safe: logs only when OAuth credentials are not configured.

```
def _configured() -> bool: return bool(GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET and GOOGLE_REFRESH_TOKEN)
```

Returns True only when all three OAuth credentials are present. Used as the gate before any real Calendar API call.

```
def _service(): from google.oauth2.credentials import Credentials from googleapiclient.discovery import build
```

Imports the Google client libraries inside the function so they are only required when Calendar is actually being used. build() constructs the Calendar API client object.

```
creds = Credentials(token=None, refresh_token=GOOGLE_REFRESH_TOKEN, ...)
```

Creates OAuth2 credentials from a refresh token. token=None means no access token yet -- the Google SDK will automatically fetch one using the refresh token on the first API call.

```
event = {"summary": ..., "start": {"date": day}, "end": {"date": day}, "attendees": [...]}
```

The Calendar event payload. "date" (not "dateTime") makes it an all-day event. attendees is a list of email dicts -- Google Calendar will email each attendee an invitation.

```
created = _service().events().insert(calendarId=KT_CALENDAR_ID, body=event, sendUpdates="all").execute()
```

Creates the event via the Google Calendar API. sendUpdates="all" triggers invitation emails to all attendees. Returns the created event object including its id.

```
def book_kt_event(case, task) -> dict: result = _create_event(case, task) if result.get("event_id")
and task.get("id"): db.table("exit_tasks").update({"kt_event_id": result["event_id"]}).eq("id",
task["id"]).execute()
Public entry point. After creating the event, writes the Calendar event id back to the exit_tasks row so there is a
permanent link between the task and its calendar entry.
```

Hub -- Orchestrator

agents/hub/supervisor.py

Agent #20 -- the top-level LangGraph that drives the full exit pipeline. Routes each case through HR -> manager gate -> IT -> compliance -> finance -> assess, with a rejection branch to escalation.

```
from langgraph.graph import END, StateGraph from typing_extensions import TypedDict
> Concept: LangGraph -- a library for building stateful, graph-based agent workflows. StateGraph is a directed graph where each node
is a function that reads and writes a shared state dict. END is a sentinel node that terminates the graph. TypedDict defines the exact
keys allowed in the state.
```

```
class SupervisorState(TypedDict): case_id: str kt_text: str | None interview_text: str | None
simulate_rejection: bool log: list[str]
The state object passed through every node in the graph. TypedDict enforces the key names and types. Every node
receives a copy of this and returns an updated copy. simulate_rejection lets the CLI trigger the rejection branch without
a human manager.
```

```
def _record(state, stage, detail) -> None: state["log"].append(detail)
db.table("agent_runs").insert({...}).execute()
Appends to the in-memory log AND inserts a row into agent_runs so the HR dashboard's Agent Activity page can show
what happened. The terminal trace alone is stdout-only and not stored anywhere.
```

```
def _activate(case_id) -> None: updated = db.table("exit_cases").update({"status": "in_progress"})\
.eq("id", case_id).eq("status", "open").execute().data
Moves the case from "open" to "in_progress". The double .eq() filter (id matches AND current status is "open") is a
forward-only guard: already-in_progress or completed cases are silently skipped, so re-running the pipeline never
downgrades a case's status.
```

```
@traced_node("Supervisor -- HR stage") def _hr_stage(state: SupervisorState) -> SupervisorState:
The HR stage node. @traced_node wraps it to print start/done/timing in the terminal trace. Every node has the same
signature: takes the state dict, returns the (updated) state dict.
```

```
checklist_generator_agent.generate(state["case_id"])
Calls the checklist agent to generate and persist the employee's HR checklist. This is a call to a compiled LangGraph
subgraph in another file -- the supervisor treats it as a single function call.
```

```
if state.get("kt_text"): kt_document_reviewer_agent.review(state["case_id"], state["kt_text"])
Only runs KT document review if kt_text was provided (CLI --kt-text flag or supervisor invocation with kt_text).
Conditional agents -- run only when their input exists.
```

```
routing = smart_routing.pick_approver(state["case_id"], "hr")
Calls the smart routing tool-node to pick the best HR approver, recording the result. smart_routing is called as a tool,
not as a graph node -- it runs inside the calling stage node.
```

```
def _route_after_manager(state: SupervisorState) -> str: return "rejected" if
state["simulate_rejection"] else "approved"
The routing function for the conditional edge after the manager gate. Returns a string key that LangGraph uses to pick
which next node to go to. The real approval comes from the database in service.py's /manager-approve route;
simulate_rejection is only for the CLI.
```

```
@traced_node("Supervisor -- escalate") def _escalate(state):
Escalation node -- only reached on a rejection. Inserts a pending task into exit_tasks with escalation_state="open" so it
appears in the HR Escalations page. Does not loop back -- the exit is stopped until a human HR person resolves it.
```

```
_graph = StateGraph(SupervisorState)
> Concept: StateGraph -- creates an empty directed graph with SupervisorState as its shared state type. Nodes and edges will be
added next.
```

```
_graph.add_node("hr", _hr_stage) _graph.add_node("manager_gate", _manager_gate) ...
```

Registers each node by name. The string name is what edges reference. The second argument is the Python function that runs when the node executes.

```
_graph.set_entry_point("hr")
```

Tells LangGraph which node to run first when the graph is invoked.

```
_graph.add_conditional_edges("manager_gate", _route_after_manager, {"approved": "it", "rejected": "escalate"})
```

> Concept: conditional edge -- after manager_gate runs, calls _route_after_manager(state) to get a string, then looks it up in the dict to find the next node. "approved" -> it node, "rejected" -> escalate node.

```
_graph.set_finish_point("assess") _graph.add_edge("escalate", END)
```

Two ways the graph ends: normal path finishes at "assess"; rejection path finishes at the END sentinel after escalation.

```
supervisor_graph = _graph.compile()
```

.compile() validates the graph structure (no disconnected nodes, valid edges) and returns a runnable object. After compile, you cannot add more nodes or edges.

```
def run_case(case_id, *, kt_text=None, interview_text=None, simulate_rejection=False) -> dict:  
    return supervisor_graph.invoke({...})
```

Public entry point. .invoke() starts the graph with the given initial state and runs nodes in sequence until it reaches a finish point. Returns the final state dict.

Spokes -- Individual Agents

agents/spokes/hr_agent.py

Agent #2 -- generates the exit checklist (LLM, by role/department) and reviews KT documents for gaps. Two compiled LangGraph subgraphs.

```
CHECKLIST_SYSTEM_PROMPT = load_prompt("hr/checklist_system.md")
```

Loads the HR checklist system prompt at module import time. This is the instruction text sent to the LLM telling it to generate a role-appropriate exit checklist. Loading at import time means the file is read once, not on every call.

```
IT_OWNED_TITLE_RE = re.compile( r"^s*(?:revoke|de-?provision|disable|...", re.IGNORECASE, )
```

> Concept: compiled regex -- re.compile() creates a reusable pattern object. Calling it once at module level is faster than re.search(pattern, text) in a loop, which re-compiles the pattern every time. This regex matches IT-owned task titles (revoke access, collect laptop) so they can be filtered OUT of manager-stage tasks.

```
class ChecklistState(TypedDict): case_id: str case: dict result: dict
```

State for the checklist subgraph. Three keys: the case ID (for DB writes), the full case row (for context to the LLM), and result (where the LLM output is stored between nodes).

```
@traced_node("HR agent -- generate checklist") def _generate_checklist(state: ChecklistState) ->  
ChecklistState: user = f"Role: {case['role_title']}\nDepartment: {case['department']}"  
state["result"] = ask_claude_json(CHECKLIST_SYSTEM_PROMPT, user)
```

First node: asks the LLM to generate a checklist. Sends only role and department -- not the full case -- so the LLM produces generic role-appropriate tasks, not case-specific ones. ask_claude_json parses the response as a JSON dict with hr_tasks and manager_tasks keys.

```
(it_owned if IT_OWNED_TITLE_RE.search(t or "") else kt_titles).append(t)
```

Sorts each manager task into either it_owned (to be dropped) or kt_titles (to be kept) by testing its title against the regex. Tasks that match the IT-owned pattern are dropped here -- not re-staged to IT, so they don't interfere with the IT agent's idempotency check later.

```
manager_rows = [ {"case_id": case_id, "stage": "manager", "title": t, "status": "pending",  
"due_date": (last_day - timedelta(days=1)).isoformat()} for t in kt_titles ]
```

Builds the database rows for KT tasks. due_date is set one day before the last working day. timedelta(days=1) is a standard-library duration object; subtracting it from a date gives the previous day.

```
inserted = db.table("exit_tasks").insert(rows).execute().data or []
```

Inserts all HR and manager task rows in a single batch insert. .data is the list of inserted rows returned by Supabase (including their auto-generated IDs). The or [] handles the case where .data is None.

```
for t in inserted_manager_rows: book_kt_event(state["case"], t)
```

After inserting the KT tasks, books a Google Calendar event for each one. `book_kt_event` writes the Calendar event ID back to the `exit_tasks` row.

```
def generate_checklist(case_id, force=False) -> dict: if not force: existing = db.table("exit_tasks").select("id").eq("case_id", case_id).in_("stage", ["hr", "manager"]).execute().data if existing: return {"skipped": True}
```

Idempotency guard: before running the graph, checks whether HR/manager tasks already exist for this case. If they do (e.g. the agent was already run or the case was seeded), returns immediately with `skipped=True` instead of creating duplicates. `force=True` bypasses this check.

```
def review_kt_document(case_id, kt_text) -> dict: return kt_review_graph.invoke({"case_id": case_id, "kt_text": kt_text, "result": {}})
```

Entry point for KT document review. Invokes the second subgraph with the raw text of an uploaded handover document. The LLM checks it for gaps and returns a summary with a list of missing topics.

agents/spokes/it_agent.py

Agent #3 -- generates the IT deprovisioning plan (LLM): what accounts to disable, what access to revoke, what assets to collect. Writes stage='it' exit_tasks.

```
SYSTEM_PROMPT = load_prompt("it/deprovisioning_system.md")
```

The system prompt tells the LLM to generate an IT deprovisioning task list using specific keyword patterns that the IT dashboard's frontend regex already understands (e.g. "laptop" for asset recovery, "SSO" for identity tasks).

```
class ItPlanState(TypedDict): case_id: str case: dict result: dict
```

Same shape as `ChecklistState` -- every agent in this repo uses a `TypedDict` state with at least `case_id`, the fetched case row, and a result placeholder.

```
user = f"Role: {case['role_title']}\nDepartment: {case['department']}" state["result"] = ask_claude_json(SYSTEM_PROMPT, user)
```

Sends only role and department to the LLM, same pattern as `hr_agent`. The LLM returns a JSON object with a "tasks" key containing a list of deprovisioning task title strings.

```
rows = [ {"case_id": case_id, "stage": "it", "title": t, "status": "pending", "due_date": last_day.isoformat()} for t in state["result"].get("tasks", []) ]
```

Builds the `exit_tasks` rows. `stage="it"` routes these tasks to the IT dashboard. `due_date` is the last working day itself (not a day before, unlike manager tasks). `.get("tasks", [])` safely returns an empty list if the LLM output was missing the key.

```
def generate_plan(case_id, force=False) -> dict: existing = db.table("exit_tasks").select("id").eq("case_id", case_id).eq("stage", "it").execute().data if existing: return {"skipped": True}
```

Same idempotency guard as `hr_agent`: returns immediately if IT tasks already exist, so re-running supervisor or `service.py /manager-approve` never creates duplicate IT tasks.

agents/spokes/finance_agent.py

Agent #4 -- deterministic finance clearance check (no LLM). Creates or updates the stage='finance' exit_tasks row based on whether prior stages are done AND dues are settled.

```
other_tasks = db.table("exit_tasks").select("status, stage").eq("case_id", case_id).in_("stage", ["hr", "manager", "it"]).execute().data or []
```

Fetches all HR, manager, and IT tasks for this case. `.in_("stage", [...])` is an SQL IN clause -- returns rows where stage matches any value in the list.

```
stages_done = bool(other_tasks) and all(t["status"] == "done" for t in other_tasks)
```

> Concept: `all()` -- returns True only if every item in an iterable is truthy. Here checks that every prior-stage task has status "done". `bool(other_tasks)` ensures there is at least one task (an empty list would make `all()` return True vacuously).

```
dues_settled = bool(case.get("finance_cleared"))
```

Reads the `finance_cleared` boolean column on `exit_cases`. This is set by the Finance dashboard's "Mark dues settled" button via an RLS-scoped RPC.

```
cleared = stages_done and dues_settled reason = None if cleared else ("dues/settlement not confirmed" if stages_done else "prior stages not complete")
```

Clearance requires BOTH conditions. The reason string is a specific, human-readable block message stored in the task title so HR can see exactly what is blocking.

```
if not finance_tasks: db.table("exit_tasks").insert({...}).execute()
```

If no finance task exists yet, inserts a new one. Otherwise falls through to the update block below. This is upsert logic written as an explicit if/else rather than a Supabase upsert -- to preserve the existing task row's ID (upsert would replace it).

```
newly_cleared = cleared and any(t["status"] != new_status for t in finance_tasks)
```

Detects the transition from pending to done -- "newly_cleared" is only True when clearance just happened, not on every re-run after it was already done. This prevents the completion email from being sent multiple times.

```
if newly_cleared and case: email_drafting_agent.completion_notice(case)
```

Sends the completion email only on the first time clearance goes from pending to done. email_drafting_agent wraps the actual send with error handling so an SMTP failure does not abort the finance check.

agents/spokes/risk_agent.py

Agent #12 -- deterministic risk scoring (no LLM). Computes risk_score, risk_level, and rehire_eligible from tenure, department criticality, interview sentiment, and task completion rate.

```
DEPARTMENT_CRITICALITY = {"Engineering": 0.9, "Sales": 0.6, ...}
```

A hardcoded lookup table mapping each department to a criticality weight (0-1). Higher means losing someone from this department is riskier. Used in the risk score formula.

```
SENTIMENT_RISK = {"negative": 1.0, "neutral": 0.5, "positive": 0.1}
```

Maps exit interview sentiment to a risk contribution. Negative sentiment (employee left unhappy) contributes more risk than positive.

```
WEIGHTS = {"tenure": 0.2, "role": 0.3, "sentiment": 0.3, "tasks": 0.2}
```

How much each factor contributes to the final score. The four weights sum to 1.0 so the result is always in [0, 1].

```
def _tenure_risk(created_at) -> float: days = (datetime.now(timezone.utc) - created).days if days < 90: return 0.9 if days < 365: return 0.6 return 0.3
```

Proxies tenure from profiles.created_at (no hire_date column exists in the schema). Short tenure = higher risk. Returns a value in [0, 1].

```
def _tasks_risk(total, done) -> float: if total == 0: return 0.0 return round((total - done) / total, 2)
```

Fraction of tasks still pending. 100% pending = 1.0 risk; all done = 0.0. round(..., 2) keeps it to two decimal places.

```
risk_score = round( tenure * WEIGHTS["tenure"] + role * WEIGHTS["role"] + sentiment * WEIGHTS["sentiment"] + tasks_risk * WEIGHTS["tasks"], 3, )
```

Weighted sum of the four factors. Each factor is multiplied by its weight so the total is a single number in [0, 1]. round(..., 3) gives three decimal places.

```
risk_level = "high" if risk_score >= 0.66 else "medium" if risk_score >= 0.4 else "low"
```

> Concept: chained ternary -- Python evaluates left to right. risk_score >= 0.66 -> "high"; else if >= 0.4 -> "medium"; else -> "low". These thresholds define the three risk bands shown in the HR dashboard.

```
rehire_eligible = (interview or {}).get("rehire_eligible", risk_level != "high")
```

Takes rehire_eligible from the exit interview if it exists; otherwise defaults to True unless risk_level is "high". (interview or {}) handles the case where interview is None.

```
db.table("exit_cases").update(state["result"]).eq("id", state["case_id"]).execute()
```

Writes risk_score, risk_level, and rehire_eligible back to the exit_cases row. These are HR-only fields -- RLS ensures no other role can read them from the base table.

agents/spokes/compliance_agent.py

Agent #13 -- deterministic compliance verification (no LLM). Checks that asset return, NDA, and access revocation are done before final clearance. Three-node graph: check -> persist -> items.

```
CHECKS = { "asset return": re.compile(r"laptop|macbook|device|headset|asset|equipment|collect", re.I), "NDA": re.compile(r"\bnda\b|non-disclosure|confidentiality", re.I), "access revoked": re.compile(r"\baccess\b|\bssso\b|\baccount\b|revoke|deprovision", re.I), }
```

Three compliance items, each with a regex pattern. A task's title is matched against these patterns to determine which item it satisfies. \b is a word boundary -- ensures "account" matches the word "account" but not "accounts" (unless the pattern also allows plural).

```
DOC_TYPES = {"asset_return": "Asset Return Form", "nda": "NDA"}
```

Maps internal item IDs to the doc_type values stored in case_documents. A validated uploaded document (OCR-checked by doc_collection.py) can satisfy an item even when no matching exit_tasks row exists.

```
def evaluate(tasks, validated_doc_types=frozenset()) -> dict:
```

> Concept: frozenset -- an immutable set (cannot be modified after creation). Used as a default argument here because mutable defaults (like [] or {}) in Python are shared between calls and can cause subtle bugs. frozenset() is safe as a default.

```
matches = [t for t in tasks if pattern.search(t.get("title", ""))] done = [t for t in matches if t.get("status") == "done"]
```

Two list comprehensions: first finds all tasks whose title matches the compliance item's pattern; second filters those to only the ones with status "done". If any are done, the item is satisfied.

```
if done or (doc_type and doc_type in validated_doc_types): continue
```

The item is satisfied if either: a matching task is done, OR a validated document of the matching doc_type was uploaded. The or allows either signal to count -- an employee who uploaded a signed NDA does not also need an exit_tasks row for it.

```
keyword_tasks = [t for t in tasks if t.get("stage") != "compliance"]
```

Filters out the compliance-stage summary task itself before running keyword checks. The summary task's title echoes the blocking reasons (e.g. "NDA: no task found"), which would self-match on every re-run -- a bug that would keep the case blocked even after being resolved.

```
db.table("compliance_checks").upsert({ "case_id": case_id, "item": item["item"], ... }, on_conflict="case_id,item").execute()
```

> Concept: upsert -- INSERT if the row doesn't exist, UPDATE if it does. on_conflict="case_id,item" means "if a row with this case_id and item already exists, update it instead of inserting a duplicate". This makes _items_node idempotent -- running it multiple times produces the same final state.

agents/spokes/exit_intel_agent.py

Agent #8 and #19 -- two subgraphs in one file. per_case: analyzes an exit interview transcript (LLM -> summary/sentiment/themes/rehire). longitudinal: finds recurring themes across all interviews and writes trend_alerts.

```
SYSTEM_PROMPT = load_prompt("interview/summary_system.md")
```

The system prompt for interview analysis. Tells the LLM to extract summary, sentiment (positive/neutral/negative), themes (list of strings), rehire_eligible (bool), and rehire_reason.

```
state["result"] = ask_claude_json(SYSTEM_PROMPT, f"Transcript:\n{state['interview_text']}")
```

Sends the full interview transcript to the LLM. The LLM returns a structured JSON with all five fields. The transcript comes either from the employee's submitted form or a CLI text file.

```
existing = db.table("exit_interviews").select("id").eq("case_id", state["case_id"]).execute() if existing.data: db.table("exit_interviews").update(row).eq("case_id", ...) else: db.table("exit_interviews").insert(row)
```

Update-or-insert pattern without upsert: checks if a row already exists, then updates or inserts accordingly. Preserves the row ID on update (important for foreign key references). The LLM-generated fields (summary, sentiment, themes, rehire_eligible) overwrite any previous values.

```
counts: Counter[str] = Counter()
```

> Concept: Counter -- a specialized dict from the collections module that counts occurrences. Counter["management issues"] += 1 increments the count for that theme. Most common themes can be found with .most_common(n).

```
for i in interviews: for theme in i.get("themes") or []: counts[theme] += 1
depts_by_theme.setdefault(theme, set()).add(dept)
```

Nested loop: for each interview, for each theme in that interview, increment the count and track which departments mentioned it. setdefault(key, default) returns the existing value if the key exists, otherwise inserts the default and returns it.

```
state["rising"] = [ {"theme": theme, "department": ..., "count": n} for theme, n in counts.items() if n >= RISING_THRESHOLD ]
```

Filters themes to only those mentioned at least RISING_THRESHOLD (2) times across all interviews. A list comprehension with an if condition at the end.

```
db.table("trend_alerts").insert(...).execute()
```

Inserts a new trend_alert row only if no matching (theme, department) row already exists. The query.execute().data check before insert is the deduplication guard.

agents/spokes/doc_collection.py

Agent #16 -- required-document tracking and real OCR validation. Downloads uploaded files from Supabase Storage, runs Tesseract OCR, and validates content with keyword rules.

```
import pytesseract from PIL import Image
```

> Concept: *pytesseract* -- Python wrapper around the Tesseract binary. *PIL (Pillow)* opens image files in Python. Together they enable real OCR: Tesseract converts an image of a document into text, Pillow handles loading the image from bytes.

```
if not shutil.which("tesseract") and os.environ.get("TESSERACT_PATH"):
```

`pytesseract.pytesseract.tesseract_cmd = os.environ["TESSERACT_PATH"]`

`shutil.which("tesseract")` checks whether the tesseract binary is on the system PATH. If not, falls back to the `TESSERACT_PATH` env var (set in `.env` pointing to the installed binary). This lets the code work on machines where Tesseract is installed but not on PATH.

```
VALIDATION_RULES = { "NDA": [{"non-disclosure", "confidential"}, {"signature", "signed"}], "Asset Return Form": [{"asset"}, {"return", "returned"}], }
```

Validation rules per document type. Each rule is a list of keyword groups; ALL groups must match at least one keyword in the OCR text. An NDA needs BOTH "non-disclosure/confidential" AND "signature/signed" -- either alone is not sufficient.

```
DATE_RE = re.compile(r"\d{1,2}[-/][A-Za-z]{3,9}[-/]\d{2,4}|\...")
```

A regex that matches common date formats (13-Sep-2026, 13/09/2026, September 13 2026). Any document that is a personal declaration should have a date; its absence is a sign the document is incomplete.

```
def validate_content(doc_type, text, employee_name=None) -> dict:
```

Pure function -- no I/O. Checks OCR'd text against `VALIDATION_RULES` for the given `doc_type`, optionally also checking that the employee's name appears in the text (identity check). Returns `{"ok": bool, "matched": [...], "missing": [...]}`.

```
file_bytes = db.storage.from_("exit-documents").download(row["file_path"])
```

Downloads the file from the "exit-documents" Supabase Storage bucket using the service key (bypasses RLS). `row["file_path"]` is the path stored in `case_documents`, e.g. "case-uuid/NDA-timestamp.png".

```
text = pytesseract.image_to_string(Image.open(io.BytesIO(file_bytes)))
```

`io.BytesIO` wraps the raw bytes as a file-like object so Pillow can open it without writing to disk. `Image.open()` parses the image; `pytesseract.image_to_string()` calls the Tesseract binary and returns the recognized text.

```
new_status = "validated" if verdict["ok"] else "rejected"
```

```
db.table("case_documents").update({"status": new_status, "validation_detail": detail}).eq("id", row["id"]).execute()
```

Updates the `case_documents` row with the validation result. "validated" means all keyword rules passed; "rejected" means at least one group was missing. The `validation_detail` string lists what matched and what was missing.

```
def validate_one(case_id, document_id) -> dict:
```

Entry point for validating a single just-uploaded document without re-running the whole batch graph. Called by `agents.service /validate-document` immediately after an employee uploads a file.

agents/spokes/smart_routing.py

Agent #11 -- picks the real approver for a stage, preferring in-department matches and skipping out-of-office profiles in favour of a delegate.

```
def select_approver(candidates, department) -> dict:
```

Pure function -- the entire routing logic. No LLM, no DB. Takes an ordered list of candidate profiles (earliest-created first) and the case's department, returns a routing decision dict.

```
dept_matches = [c for c in candidates if department and c.get("department") == department] pool = dept_matches or candidates
```

Prefers candidates whose department matches the case's department. If none match, falls back to the full candidate list. "dept_matches or candidates" is Python's way of saying "use dept_matches if it is non-empty, otherwise candidates".

```
available = [c for c in pool if not c.get("out_of_office")]
```

Filters the pool to only candidates who are NOT out of office. `out_of_office` is a boolean column on profiles (added in migration 0012).

```
if available: chosen = available[0] is_delegate = chosen["id"] != pool[0]["id"]
```

If anyone is available, pick the first one. `is_delegate` is True when the first available person is not the first in the pool -- meaning the primary was skipped due to OOO and a delegate was chosen.

```
chosen = pool[0] return {..., "all_ooo": True, "reason": "every candidate is out_of_office..."}
```

Fallback: if everyone is OOO, pick the primary anyway and flag `all_ooo=True`. The pipeline continues -- a fully blocked team does not crash the exit case, it just warns.

```
candidates = db.table("profiles").select("id, full_name, email, department, out_of_office, created_at") .eq("role", state["stage"]).order("created_at").execute().data
```

Fetches all profiles with the matching role (e.g. "hr" for the HR stage) ordered by `created_at`. The seed data ensures the primary (non-delegate) account was created first, so index 0 is always the primary.

agents/spokes/email_drafting_agent.py

Agent #6 -- routing point for all outbound emails. Wraps notifications.py with error handling and audit logging to agent_runs.

```
def _drafted(template, case_id, fn, *args, **kwargs) -> dict:
```

Core wrapper. `fn` is the actual send function from notifications.py. `*args` and `**kwargs` pass through all arguments to `fn`.

If `fn` raises (e.g. SMTP failure), the exception is caught, logged to `agent_runs` as `status="failed"`, and a failure dict is returned -- the pipeline continues.

```
try: result = fn(*args, **kwargs) except Exception as exc: error = f"{type(exc).__name__}: {exc}"
_record_email(case_id, template, None, error=error) return {"sent": False, "logged": False, "to":
None, "error": error}
```

> Concept: `try/except` -- catches exceptions. `type(exc).__name__` is the exception class name (e.g. "ConnectionRefusedError"). This ensures an email send failure is audited but never crashes the calling agent.

```
def _status(outcomes) -> str: if any(o.get("sent") for o in outcomes): return "smtp_accepted" if
any(o.get("logged") for o in outcomes): return "dev_logged" return "failed"
```

> Concept: `any()` -- returns True if at least one item in an iterable is truthy. Determines overall send status from a list of per-recipient outcomes (a KT reminder goes to both employee and manager, so there can be multiple outcomes).

```
db.table("agent_runs").insert({"case_id": ..., "stage": "email_drafting", "status": status,
"metadata": metadata}).execute()
```

Records the email attempt to `agent_runs` so HR can see in the Agent Activity page what emails were sent, to whom, and whether they succeeded. `metadata` includes the template name, recipients, and per-recipient outcomes.

agents/spokes/it_deprovisioning_agent.py

Agent #18 -- execute/verify/audit wrapper around the IT deprovisioning plan. Uses `MockITAdapter` (no real Okta/Jamf integration). Idempotent: skips already-executed tasks.

```
ASSET_RE = re.compile(r"laptop|macbook|device|headset|card|asset|collect", re.I) IDENTITY_RE =
re.compile(r"sso|identity|account", re.I) REPO_RE = re.compile(r"repo|repository|source|git", re.I)
```

Three regexes that classify an IT task's action type from its title. These mirror the exact keyword patterns the IT dashboard's frontend uses to sort tasks into visual categories -- they must stay in sync.

```
def _verify_execution(task, execution) -> dict: expected = _classify_action(task["title"]) verified
= execution.get("ok") is True and execution.get("action_type") == expected
```

Re-derives the expected action type independently from the task title and cross-checks it against what the adapter reported. A task that returned `ok=True` but with the wrong `action_type` does NOT verify -- guards against the adapter executing the wrong action.

```
class MockITAdapter: timeout_seconds: float = 5.0 max_retries: int = 1
```

> Concept: class -- a blueprint for objects. `MockITAdapter` has two class-level attributes (`timeout_seconds`, `max_retries`) shared by all instances. `MockITAdapter` is explicitly documented as a mock -- no real IT system is called.

```
with ThreadPoolExecutor(max_workers=1) as pool: result = pool.submit(self._run,
task).result(timeout=self.timeout_seconds)
```

> Concept: `ThreadPoolExecutor` -- runs a function in a background thread. `.result(timeout=N)` waits up to N seconds for the thread to finish. If it times out, raises `FuturesTimeoutError`. This enforces a real timeout on the simulated IT provider call.

```
for attempt in range(self.max_retries + 1):
```

Loop runs max_retries+1 times (once for the initial try, once for the retry). If the provider fails on both attempts, the last error result is returned.

```
already_executed = {r["metadata"]["task_id"] for r in audited if r.get("status") == "verified" ...}
```

> Concept: set comprehension -- {expression for item in iterable if condition} builds a set. Collects the task IDs of tasks that already have a "verified" audit row. Only verified tasks are skipped on re-run -- a previously failed task stays retryable.

Analytics Agents

agents/analytics/analytics_agent.py

Agent #14 -- aggregates exit_cases/exit_tasks statistics in Python (no LLM for math), then asks the LLM to write a narrative summary. Three-node graph: aggregate -> narrate -> persist.

```
from collections import Counter
```

> Concept: Counter -- a subclass of dict that counts things. Counter(c["department"] for c in cases) counts how many cases each department has in a single pass.

```
by_department = Counter(c["department"] for c in cases)
```

> Concept: generator expression -- (expression for item in iterable) creates a lazy iterator that Counter consumes. More memory-efficient than building a full list first.

```
pending_by_stage = Counter(t["stage"] for t in tasks if t["status"] != "done")
```

Counts pending tasks grouped by stage. The if condition filters out completed tasks before counting. This tells HR which stage has the most bottleneck work.

```
state["narrative"] = ask_claude(SYSTEM_PROMPT, f"Stats:\n{state['stats']}")
```

Only the narrative generation uses the LLM. Passing the pre-computed stats dict as a string means the LLM never does arithmetic -- it just writes a plain-English description of numbers we already computed.

```
db.table("analytics_insights").insert({"narrative": state["narrative"], "stats": state["stats"],  
"agent_type": "dashboard_insights"}).execute()
```

Stores both the LLM narrative and the raw stats dict. The HR dashboard reads the latest analytics_insights row to display insights. agent_type distinguishes this agent's rows from the other analytics agents that write to the same table.

agents/analytics/sla_escalation.py

Agent #9 -- scans for pending tasks overdue by >=5 days and sends escalation emails to the responsible approver (HR or manager), naming the blocker and downstream impact.

```
THRESHOLD_DAYS = 5
```

The number of days past due_date a task must be before it triggers an escalation. Tasks overdue by 1-4 days are handled by notifications.check_overdue_and_notify (a softer reminder); this agent is for serious breaches.

```
def find_breaches(tasks, cases, profiles, today) -> list[dict]:
```

Pure function -- all database reads happen in the calling node; this function only does arithmetic and lookups on the data already fetched. Makes it testable without a database connection (the self-check at the bottom uses this).

```
due = t["due_date"] if isinstance(t["due_date"], date) else date.fromisoformat(t["due_date"])
```

Supabase can return dates as either Python date objects or ISO-format strings depending on context. This normalizes both to a date object before subtraction.

```
days_overdue = (today - due).days
```

Subtracting two date objects gives a timedelta; .days extracts the integer number of days. If today is after due, this is positive (overdue). The if condition below filters to >= THRESHOLD_DAYS.

```
blocker = profiles.get(case.get("hr_id"), {}).get("full_name") or "HR"
```

Looks up the HR person's name from the profiles dict. Nested .get() calls handle the case where hr_id is None (returns {}) or the profile is not in the dict (returns "HR" as a fallback name).

agents/analytics/attrition_agent.py

Agent #23 -- predictive attrition using department-level proxies (no individual hire-date data). Identifies at-risk departments from high average exit risk_score and trend_alerts, then writes an LLM narrative.

```
scores_by_dept: dict[str, list[float]] = defaultdict(list)
```

> Concept: defaultdict -- a dict that automatically creates a default value when you access a missing key. defaultdict(list) means scores_by_dept["Engineering"] starts as [] instead of raising KeyError. Used to accumulate risk scores per department.

```
high_risk_depts = {d for d, scores in scores_by_dept.items() if sum(scores) / len(scores) >= HIGH_RISK_AVG}
```

Set comprehension: computes the average risk_score for each department and includes the department in the set if the average is >= HIGH_RISK_AVG (0.6). Departments without any scores are excluded by scores_by_dept.items().

```
alert_depts = {a["department"] for a in alerts if a.get("department") and a.get("severity") != "low"}
```

Departments flagged by trend_alerts with medium or high severity (recurring negative exit-interview themes).

Combined with high_risk_depts to form the full at-risk signal.

```
at_risk_employees = [e for e in employees if e.get("department") in signals]
```

"signals" is the dict of at-risk departments. Checking "in signals" uses the dict's keys -- a fast O(1) lookup. This gives a count of current employees in at-risk departments (not a list of their names, to avoid privacy concerns).

Service and Entry Points

agents/service.py

Local HTTP bridge on port 8787. The frontend calls these 9 routes to trigger pipeline actions that the Supabase Edge Functions cannot reach (Python-only pipeline on the dev machine).

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
```

> Concept: stdlib HTTP server -- Python's built-in web server, used here instead of Flask/FastAPI because neither is a project dependency. ThreadingHTTPServer handles each request in its own thread so one slow agent call does not block other requests.

```
PORT = 8787 ALLOWED_ORIGIN = "http://localhost:5173"
```

The agent service runs on port 8787; the React frontend runs on port 5173. ALLOWED_ORIGIN is used in CORS headers to allow the frontend to call this service (browsers block cross-origin requests by default).

```
def activate_case(case_id) -> dict: case = db.table("exit_cases").select("*").eq("id", case_id).single().execute().data
```

Fetches the full case row. .single() tells Supabase to expect exactly one row and raise PGRST116 if none match. Note: other functions in this file use a plain .execute() with a None check instead, because .single() raises on zero rows.

```
checklist_result = hr_agent.generate_checklist(case_id)
```

Runs the HR checklist agent for this case. This is the only place this call is made on the browser-triggered path -- the supervisor graph's HR stage does the same thing on the CLI path.

```
def manager_approve(case_id) -> dict:
```

The browser-triggered equivalent of the supervisor graph's approved-manager-gate branch. Checks that all KT tasks are genuinely done in the database before advancing -- the human click already happened, this just reads it back and proceeds.

```
escalation_ids = {t["id"] for t in manager_tasks if (t.get("title") or "").startswith("Escalated")}
```

Finds escalation rows by their title prefix. Escalation rows are manager-stage rows but should not count as pending KT work -- they are resolved through HR, not by completing KT.

```
class Handler(BaseHTTPRequestHandler): def _cors(self) -> None:
```

```
self.send_header("Access-Control-Allow-Origin", ALLOWED_ORIGIN)
```

> Concept: class inheritance -- Handler extends BaseHTTPRequestHandler, inheriting its HTTP parsing. _cors() adds CORS headers to every response so the browser accepts the reply from a different port.

```
def do_OPTIONS(self) -> None:
```

> Concept: CORS preflight -- before sending a POST with custom headers, the browser first sends an OPTIONS request. If the server does not respond with the right CORS headers, the browser blocks the actual request. do_OPTIONS handles this preflight.

```
routes = { "/activate-exit": lambda body: activate_case(body["case_id"]), ... }
```

> Concept: lambda -- an anonymous function. lambda body: activate_case(body["case_id"]) is equivalent to def f(body): return activate_case(body["case_id"]). The routes dict maps URL paths to handler lambdas, keeping do_POST clean.

```
length = int(self.headers.get("Content-Length", 0)) body = json.loads(self.rfile.read(length) or b"{}")
```

Reads the request body. Content-Length tells us how many bytes to read; self.rfile is the incoming byte stream. json.loads() parses the JSON; the or b"{}" handles empty bodies.

```
server = ThreadingHTTPServer(("localhost", PORT), Handler) server.serve_forever()
```

Creates and starts the HTTP server. serve_forever() runs an infinite loop, accepting and dispatching one thread per request until the process is killed.

Supabase Edge Functions (Deno/TypeScript)

supabase/functions/submit-resignation/index.ts

Edge Function: creates the exit_cases row when an employee submits their resignation form. Runs in Supabase's cloud. Identity is derived server-side from the session JWT.

```
const DEPARTMENT_TITLES: Record<string, string> = { Engineering: "Software Engineer", ... }
```

> Concept: TypeScript Record<K,V> -- a dict type where every key is type K and every value is type V. Maps department names to default role titles because profiles has no role_title column of its own.

```
const admin = createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, { auth: { autoRefreshToken: false, persistSession: false } })
```

Creates a service-role Supabase client (bypasses RLS). autoRefreshToken and persistSession are set to false because Edge Functions are stateless -- there is no session to persist between requests.

```
Deno.serve(async (req) => {
```

> Concept: Deno.serve -- Deno's built-in HTTP server. The function receives each request as a Request object and must return a Response. Supabase deploys this as a serverless function.

```
const authHeader = req.headers.get("Authorization") const authed = createClient(SUPABASE_URL, SUPABASE_ANON_KEY, { global: { headers: { Authorization: authHeader } } })
```

Forwards the caller's session JWT to a new anon-key client. This client's requests run as the calling user -- so getUser() returns the logged-in employee's identity, not any identity from the request body.

```
const { data: { user } } = await authed.auth.getUser()
```

> Concept: destructuring assignment -- extracts user from the nested object { data: { user } }. getUser() verifies the JWT against Supabase Auth and returns the authenticated user record.

```
const { data: profile } = await admin.from("profiles").select("full_name, email, employee_id, department, role").eq("id", user.id).single()
```

Reads the employee's profile using the service key. Checks role === "employee" to prevent HR/manager accounts from submitting a resignation through this endpoint.

```
const { data: existing } = await admin.from("exit_cases").select("id").eq("employee_id", profile.employee_id).maybeSingle()
```

> Concept: maybeSingle() -- like .single() but returns null instead of throwing when no row is found. Used here because an existing case is a normal case (idempotent re-submission), not an error.

```
if (existing) return json({ ok: true, case: existing })
```

If the employee already has an exit case, return it immediately without creating a duplicate. The frontend then proceeds to trigger the agent pipeline.

```
const { data: created, error } = await admin.from("exit_cases").insert({...}).select().single().insert().select().single()
```

insert().select().single() inserts a row and returns the newly created row in one call. If error is non-null, the insert failed (e.g. a constraint violation).

supabase/functions/ask/index.ts

Edge Function: the RAG assistant. Embeds the employee's question, retrieves the top-k matching policy chunks, sends them to the LLM with the question, returns a structured answer with citations.

```
const SIMILARITY_THRESHOLD = 0.35
```

Below this cosine similarity score the top retrieved chunk is not actually relevant to the question (the user probably asked something off-topic). Still let the LLM reply, but don't cite unrelated sections.

```
function looksLikeRefusal(answer: string): boolean { ... }
```

Detects when the LLM refused in prose instead of setting scope="refused" in its JSON. Normalizes typographic apostrophes (U+2019) to ASCII apostrophes before the regex test, because the Azure model uses the fancy

apostrophe in contractions like "don't".

```
async function embed(input: string): Promise<number[]> { const res = await
fetch(`${PORTKEY_BASE_URL}/v1/embeddings`, ...)
```

> Concept: async/await -- await pauses execution until the Promise (the fetch result) resolves. fetch() makes an HTTP request and returns a Promise. The embedding endpoint converts the question text into a 1536-dimension vector of floats.

```
const { data: chunks, error } = await db.rpc("match_exit_docs", { query_embedding: queryEmbedding,
match_count: 4 })
```

Calls the match_exit_docs SQL function (defined in 0003_rag.sql) via Supabase RPC. Passes the question's embedding vector; returns the 4 closest policy chunks by cosine similarity.

```
const context = chunks .map((c) => `[${c.section} || c.source]`)\n${c.content}`) .join("\n\n")
```

Formats the retrieved chunks into a single context string. Each chunk is prefixed with its section label in square brackets -- the LLM is told to report which sections it used by copying these labels.

```
function parseReply(raw: string): { answer: string; sections: string[]; scope: Scope | null }
```

Parses the LLM's JSON response, tolerating common failure modes: markdown fences around the JSON, unescaped newlines inside strings. Falls back to regex extraction if JSON.parse fails.

```
const used = new Set(sections.map((s) => s.trim().toLowerCase()).filter(Boolean)) const matched =
chunks.filter((c) => used.has(String(c.section ?? c.source).trim().toLowerCase()))
```

> Concept: Set -- a collection with no duplicates and O(1) lookup. Maps cited section names to lowercase for case-insensitive matching, then finds the chunks whose section labels the model actually cited.

```
return new Response(JSON.stringify({ answer, sources, refused, general }), { headers: {
...CORS_HEADERS, "Content-Type": "application/json" } })
```

Returns the final structured response. The "..." spread operator copies CORS_HEADERS into a new object and adds Content-Type. The frontend uses the refused and general flags to show appropriate UI (forward-to-HR button on refusal, disclaimer on general guidance).

SQL Migrations

supabase/migrations/0001_schema.sql

Base schema: creates the five core tables (profiles, exit_cases, exit_tasks, exit_interviews, trend_alerts) and the exit_docs RAG store with its pgvector column.

```
create extension if not exists vector;
```

> Concept: PostgreSQL extension -- adds extra functionality to the database. The "vector" extension (pgvector) adds a vector data type and vector-similarity operators. "if not exists" makes this safe to run multiple times.

```
create table if not exists profiles ( id uuid primary key references auth.users(id) on delete
cascade, role text not null check (role in ('employee','hr','manager','it')), ...
```

profiles stores one row per auth user. id references auth.users -- when Supabase deletes an auth user, cascade deletes the profile too. The CHECK constraint enforces that role is always one of four valid values. uuid primary key uses a 128-bit random ID.

```
create table if not exists exit_cases ( id uuid primary key default gen_random_uuid(), risk_level
text check (risk_level in ('low','medium','high')), risk_score numeric, rehire_eligible boolean,
exit_cases has HR-only assessment columns at the bottom (risk_level, risk_score, rehire_eligible). The comment
"HR-only below this line" documents the security boundary enforced by RLS in 0002_rls.sql.
```

```
create table if not exists exit_tasks ( stage text not null, status text not null default 'pending'
check (status in ('pending','done')), kt_event_id text,
exit_tasks has no enum for stage (stages are "hr", "manager", "it", "compliance", "finance") because the list grew during
development. kt_event_id stores the Google Calendar event ID written by calendar_booking.py.
```

```
create table if not exists exit_docs ( embedding vector(1536)
```

> Concept: vector(1536) -- a pgvector column that stores a 1536-dimensional floating-point vector. 1536 is the dimension of the OpenAI text-embedding-3-small model. The dimension must match exactly or inserts fail.

```
create index if not exists idx_exit_docs_embedding on exit_docs using hnsu (embedding
vector_cosine_ops);
```

> Concept: HNSW index -- Hierarchical Navigable Small World, a graph-based approximate nearest-neighbor index. It finds the most similar vectors very fast (milliseconds) without checking every row. vector_cosine_ops means it measures cosine distance.

supabase/migrations/0002_rls.sql

Row-Level Security: the access control model. Implements the non-negotiable that HR-only fields are never exposed to employees, managers, or IT -- enforced in Postgres, not just hidden in the UI.

```
create or replace function public.app_current_role() returns text language sql stable security
definer set search_path = ''
```

> Concept: security definer function -- runs with the permissions of the function's OWNER (the postgres superuser), not the calling user. Needed here because the function reads from profiles, which is itself under RLS. Empty search_path prevents an attacker from shadowing the profiles table with their own object.

```
as $$ select p.role from public.profiles p where p.id = auth.uid(); $$;
```

> Concept: \$\$... \$\$ -- the function body delimiter in PostgreSQL. auth.uid() is a Supabase-provided function that returns the UUID of the currently authenticated user from their JWT. The function returns the role string ("hr", "employee", etc.).

```
alter table public.profiles enable row level security; alter table public.exit_cases enable row
level security;
```

> Concept: row-level security (RLS) -- when enabled, a table with no matching policy returns ZERO rows to any query, even SELECT *. This is the "default deny" posture. Policies below then selectively grant access.

```
create policy exit_cases_hr_select on public.exit_cases for select to authenticated using
(public.app_current_role() = 'hr');
```

> Concept: RLS policy -- a WHERE clause added automatically to every query. This policy says: for SELECT queries on exit_cases, only return rows where the current user's role is "hr". Employees and managers have NO policy here -- they read 0 rows from the base table.

```
create or replace view public.employee_exit_view with (security_invoker = false) as select ec.id,
ec.employee_name, ec.department, ec.role_title, ec.last_working_day from public.exit_cases ec join
public.profiles p on p.employee_id = ec.employee_id where p.id = auth.uid();
```

> Concept: security_invoker = false (security definer view) -- the view runs with the view OWNER's permissions, not the querying user's. This lets it see past exit_cases's HR-only RLS policy, but the SELECT list only exposes the five safe columns -- never risk_level, risk_score, or rehire_eligible. The WHERE p.id = auth.uid() ensures each employee only sees their own case.

```
create policy exit_tasks_it_select on public.exit_tasks for select to authenticated using
(public.app_current_role() = 'it' and stage = 'it');
```

IT can read exit_tasks rows, but only rows where stage = "it" (their own work). An IT user cannot read HR-stage or finance-stage tasks, even if they know the case_id.

```
comment on table public.exit_docs is 'RAG chunk store. RLS enabled with NO anon/authenticated policy
on purpose...'
```

exit_docs has RLS enabled but zero SELECT policies. This means nobody can query it directly -- only the service-role /ask Edge Function (which bypasses RLS) can read it. This makes /ask a real security boundary, not just a convenience wrapper.

supabase/migrations/0003_rag.sql

The RAG search function: match_exit_docs. Takes a query embedding vector, returns the top-k most similar chunks by cosine similarity.

```
create or replace function match_exit_docs( query_embedding vector(1536), match_count int default 4
)
```

The function accepts the embedded question vector (same 1536 dimensions as the stored chunk embeddings) and returns up to match_count rows. This is called from the /ask Edge Function via db.rpc("match_exit_docs", {...}).

```
returns table (id bigint, source text, section text, content text, similarity double precision)
```

> Concept: table-returning function -- instead of returning a single value, this function returns rows with named columns. Supabase exposes this as an RPC endpoint.

```
1 - (embedding <=> query_embedding) as similarity
```

> Concept: cosine distance (<=> operator from pgvector) measures the angle between two vectors. It returns 0 for identical vectors, 2 for opposite. Subtracting from 1 converts it to cosine SIMILARITY: 1.0 for identical, -1 for opposite. Higher is better.

```
order by embedding <=> query_embedding limit match_count;
```

ORDER BY distance ascending means the closest (most similar) chunk comes first. LIMIT returns only the top match_count results (default 4). pgvector uses the HNSW index to do this efficiently without scanning all rows.

```
-- Deliberately NOT security definer. It runs with the caller's rights, so it -- inherits exit_docs'
RLS.
```

Critical design decision: if this were security definer, any authenticated user could call it and bypass exit_docs's RLS to read policy documents directly. By keeping it security invoker (the default), the caller's RLS applies -- the anon key gets 0 rows.

Frontend -- React + Vite

src/main.jsx

React entry point. Mounts the root <App> component inside React StrictMode and imports global CSS.

```
import { StrictMode } from 'react' import { createRoot } from 'react-dom/client'
> Concept: React -- a JavaScript library for building UIs. StrictMode enables extra runtime warnings during development. createRoot() mounts the React app into a DOM element.
```

```
import '@tabler/icons-webfont/dist/tabler-icons.min.css' import './styles/tokens.css' import './styles/dashboards.css'
> Concept: CSS import in JavaScript -- Vite processes these imports and bundles the CSS into the built app. @tabler/icons loads icon fonts; tokens.css defines CSS custom properties (variables); dashboards.css has layout styles.
```

```
createRoot(document.getElementById('root')).render( <StrictMode><App /></StrictMode> )
document.getElementById('root') finds the <div id='root'> in index.html. createRoot().render() tells React to take ownership of that DOM node and render the component tree inside it.
```

src/App.jsx

Root component: handles auth state, fetches the user's role from Supabase, and renders either the login page, a loading state, or the role-appropriate dashboard.

```
const [session, setSession] = useState(undefined)
> Concept: useState -- React hook that declares a piece of state. The first value (session) is the current state; the second (setSession) is the setter function. Initial value undefined means 'still loading' -- distinct from null (no session) or a real session object.
```

```
useEffect(() => { supabase.auth.getSession().then(({ data }) => setSession(data.session)) const { data: sub } = supabase.auth.onAuthStateChange((_event, s) => setSession(s)) return () => sub.subscription.unsubscribe() }, [])
> Concept: useEffect with [] -- runs once after the component first mounts. Fetches the current session and subscribes to auth state changes (login/logout events). The return function is a cleanup: it unsubscribes when the component unmounts to prevent memory leaks.
```

```
supabase.from('profiles').select('role').eq('id', session.user.id).single().then(({ data }) => setRole(data?.role ?? null))
After a session exists, queries the profiles table for the user's role. data?.role is optional chaining: returns data.role if data is not null/undefined, otherwise undefined. ?? null converts undefined to null.
```

```
if (session === undefined || (session && role === null)) return null
While loading (session is undefined, or session exists but role not yet fetched), render nothing. This prevents a flash of the login page while auth state is resolving.
```

```
const activeRole = ROLE_SWITCHER_ENABLED && devRole ? devRole : role
In dev mode with the role switcher enabled, devRole overrides the real database role. In production (ROLE_SWITCHER_ENABLED is false), activeRole always equals role -- the real role from the database.
```

src/routes/AppRoutes.jsx

URL routing: maps paths to role-guarded components. Each role has its own nested route tree. Unknown or wrong-role paths redirect to the correct dashboard.

```
import { Routes, Route, Navigate } from 'react-router-dom'
> Concept: React Router -- the standard routing library for React SPAs. Routes is the container; Route maps a URL path to a component; Navigate performs a redirect.
```

```
<Route path="/employee" element={role === "employee" ? <EmployeeLayout session={session} /> : <Navigate to={`/${role}`} replace />} />
> Concept: ternary in JSX -- the element prop is either the real layout (if role matches) or a redirect (if not). replace means the redirect replaces the current history entry so the back button doesn't loop back to the wrong-role URL.
```

```
<Route index element={<Employee.Dashboard />} />
```

> Concept: index route -- matches the parent path exactly (no extra segment). When the URL is /employee with nothing after, render the Dashboard component.

```
<Route path="" element={<Navigate to="" replace />} />
```

> Concept: catch-all route -- path="" matches any URL that doesn't match the more specific routes above. Redirects back to the layout's root (the empty string relative path).

```
<Route path="" element={<Navigate to={`/${role}`} replace />} />
```

The outermost catch-all: if no role-specific route matched at all (e.g. someone typed /unknown), redirect to the user's dashboard (e.g. /hr, /employee).

src/lib/supabase.js

Creates and exports the single shared Supabase anon-key client used by all frontend code.

```
import { createClient } from '@supabase/supabase-js' export const supabase = createClient(
import.meta.env.VITE_SUPABASE_URL, import.meta.env.VITE_SUPABASE_ANON_KEY )
```

> Concept: import.meta.env -- Vite replaces VITE_* variables at build time from .env.local. The anon key is public (it is embedded in the frontend bundle) but safe because RLS policies restrict what it can access. Never use the service key here.

Scripts

scripts/ingest_docs.js

One-time (re-runnable) script: reads exit_policy.md, chunks it by section, embeds each chunk via the Portkey embeddings API, inserts into exit_docs. Run once after changing the policy document.

```
process.loadEnvFile()
```

Loads the .env file into process.env. Node.js 20.6+ built-in equivalent of dotenv. Reads SUPABASE_SERVICE_KEY and PORTKEY_* vars needed for the service-key client and embeddings API.

```
function chunkText(text) { const paragraphs = text.split(/\n\s*\n/).map((p) =>
p.trim()).filter(Boolean) const chunks = [] for (const p of paragraphs) { if (chunks.length === 0 ||
/^S.[\d.]+\s+/.test(p)) chunks.push(p) else chunks[chunks.length - 1] += `\n\n${p}` }
```

Splits the markdown document on blank lines, then groups paragraphs by section heading (lines starting with S.n.n).

Each S.-headed section becomes one chunk; continuation paragraphs are appended to the current chunk. This keeps each policy section whole.

```
function sectionOf(chunk) { const match = chunk.match(/^S.[\d.]+\s+(.+)$/m) return match ?
match[1].trim() : null }
```

Extracts the human-readable section title (e.g. 'Final settlement timeline') from the first matching S.n.n heading line in the chunk. Stored as case_documents.section and shown as citations in the /ask response.

```
async function embed(input) { const res = await fetch(`${portkeyUrl}/v1/embeddings`, { body:
JSON.stringify({ model: embedModel, input }) }) }
```

Calls the Portkey embeddings endpoint. embedModel is the PORTKEY_VIRTUAL_KEY which routes to text-embedding-3-small. Returns a 1536-float array.

```
const { error: delErr } = await db.from('exit_docs').delete().eq('source', SOURCE)
```

Deletes all existing rows for this source file before re-inserting. This makes the script idempotent: running it again after updating exit_policy.md replaces all chunks cleanly instead of accumulating duplicates.

```
if (embedding.length !== 1536) throw new Error(`embedding dim mismatch: got ${embedding.length},
expected 1536`)
```

Validates the embedding dimension before inserting. If the model was changed and returns a different dimension, the insert would fail with a pgvector error anyway -- this early check gives a clearer message.

End of ExitAI Code Line-by-Line walkthrough. Every file covered reflects the actual code read from the repository on 2026-09-21.